Does an Executable Validator Reduce Semantic Failures in LLM-Generated Network Configuration Changes?

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We report a fully preregistered paired experiment (CNSM2026-A-prereg-v1) that tested whether inserting an executable emulator validator into a generate→validate→repair pipeline reduces semantic/network-level failures versus an LLM-only suggestion workflow. Using a deterministic paired design with N=120 intent→configuration tasks (40 per topology class: edge, core, leaf–spine) we produced one shared initial candidate per task via gpt-5-mini (model_calls_used = 120) and evaluated the identical candidate under both conditions. The guarded pipeline used deterministic_netops_validator_v5 (validator_version = 5.0) with at most one repair iteration per guarded episode. Observed outcomes: baseline successes = 120/120; guarded successes = 120/120; paired contingency: n_11=120, n_10=0, n_01=0, n_00=0. Exact two-sided McNemar p = 1.0; paired-difference estimate = 0.00; paired-bootstrap-percentile 95% CI = [0.00, 0.00] (10,000 resamples, seed = 1729). Validator traces record repair_applied = false for every guarded episode. We expand methodological detail, representative diagnostics grounded in archived validator traces and task manifests, reproducibility guidance tied to the preregistered artifacts, and discuss construct/external validity limits and design recommendations for future confirmatory studies.**

I. Introduction

Large language models (LLMs) promise to convert operator intent into actionable network configuration commands and thereby improve NetOps productivity. Syntactically plausible command sequences can nonetheless violate temporal and stateful workflow invariants (for example, changing MTU without an enforced shutdown or performing an unsafe uplink switchover) and thus produce semantic failures visible only in closed-loop execution. Prior work argues that integrating deterministic verification, emulation, or veto layers—implemented as generate→validate→repair pipelines or veto/guard layers—reduces the risk of such semantic failures when generators err (see recent benchmarks and surveys). Measuring the marginal contribution of executable validators therefore requires experiments that expose stateful, temporal, and group-level invariants rather than token- or syntax-level correctness alone.

We executed a fully preregistered paired confirmatory experiment (CNSM2026-A-prereg-v1) designed to isolate the marginal effect of an executable emulator validator and a bounded repair policy on identical generator outputs. The core research question was: does integrating an executable network emulator as a validator in a generate→validate→repair pipeline reduce semantic/network-level failures when enacting LLM-generated configuration changes compared to an LLM-only suggestion workflow? To attribute any differences exclusively to the validator/repair subsystem we used a shared-initial-candidate paired design: for each intent→configuration task we recorded a single generator output (gpt-5-mini) and scored that identical candidate under both baseline (LLM-only) and guarded (validator±repair) conditions. The execution produced a degenerate, ceiling outcome; this paper reports the numerical results, interprets their methodological meaning, and derives practical design recommendations for confirmatory evaluations that seek to exercise repair behavior.

II. Related Work

Executable emulation and closed-loop benchmarks have repeatedly shown that token- or syntax-level metrics miss important failure classes—order-dependent invariants, make-before-break violations, group-availability breaches—that only appear under stateful execution [https://arxiv.org/abs/2604.22513; https://arxiv.org/abs/2608.23179; https://arxiv.org/abs/2604.18233]. Tool-augmented agent architectures and bounded-autonomy guardrails (veto layers, canaries, generate→validate→repair) are advocated across surveys and prototypes as practical safety measures for agentic NetOps deployments [https://openalex.org/W7161354925; 10.1145/3771567].

Two methodological lessons from that literature motivated our design. First, comparisons that generate separate candidates per condition conflate generator sampling variance with verification efficacy; observed differences can be partially driven by which candidate the generator happened to produce. Second, paired shared-generation designs cleanly isolate marginal effects of verification/repair but can yield ceiling outcomes when the generator is well calibrated for the task distribution. Our preregistered paired-binary design trades off internal validity (isolation of the validator effect) against the risk that repairs are never exercised. The present study makes that trade-off explicit and provides artifact-grounded diagnostics to explain a ceiling result without contradicting prior demonstrations that validators reduce failures when generators err.

III. Methodology

Preregistration and confirmatory plan. The study was preregistered (CNSM2026-A-prereg-v1) with a deterministic

paired-binary design. Planned sample: N=120 paired tasks deterministically partitioned into three topology classes (edge, core, leaf–spine) of 40 tasks each. The primary estimand was the paired_success_rate_difference_guarded_minus_baseline; the prespecified confirmatory test was the exact two-sided McNemar test. A paired-bootstrap-percentile 95% CI with 10,000 resamples and seed=1729 was prespecified. The preregistration fixed exclusion rules, a repair budget (<=1 repair per guarded episode), deterministic reset controls, contamination checks, and sensitivity analyses.

Task generation and payload. Tasks were produced by deterministic_netops_task_generator_v5. Each manifest entry encoded initial_state, expected_final_state, a natural-language intent, an explicit required_sequence (ordered operations), and difficulty metadata (changed_interface_count, dependency_rule_count, required_command_count, level). Workflow_policies (must_be_down_for_fields, final_group_constraints, make-before-break patterns) were expressed in the task payload and enforced by the validator as executable checks rather than heuristic text matches. Representative required_command_count sampled values ranged from 2 to 6; difficulty levels include medium, hard, and very_hard. The task-manifest artifacts are archived and include per-task required_sequence and workflow_policies used by the validator during scoring.

Shared-generation paired design and generator configuration. For each pair we issued one shared generation call to the declared model (gpt-5-mini) and reused that identical shared candidate for baseline and guarded scoring. Model accounting: model_calls_used = 120, hosted_model_call_event_count = 120, cache_miss_count = 120, cache_hit_count = 0. The model configuration during the run requested gpt-5-mini with max_output_tokens = 1500, maximum_attempts_per_call = 1, provider = openai_responses, and reasoning_effort = minimal. These choices (single generation per pair, minimal reasoning effort) were declared in the preregistration and remove generator variance across paired comparisons so any discordance derives only from validator/repair actions.

Validator and repair semantics. The guarded pipeline invoked deterministic_netops_validator_v5 (validator_version = 5.0). For each guarded episode the validator (a) executed the shared candidate in a deterministic emulator seeded from the task's initial_state, (b) enforced the task-encoded workflow_policies as executable checks, and (c) emitted a structured validator_trace with fields such as validation_before, validation_after, observed_touched_field_count, parsed_command_count, required_command_count, violation_count, violation_codes, and repair_applied. The preregistered repair policy permitted at most one repair iteration when validator.violation_count > 0; repair-call traces would be archived if executed. In the run validator.violation_count = 0 and validator.repair_applied = false for all guarded episodes, so no repair-provider-call artifacts were produced.

Instrumentation and contamination controls. The harness enforced deterministic emulator resets between tasks, container checksum verification, and provider-call audits. Contamination checks included duplicate-output detection and cross-condition cache-key reuse detection; none were triggered. Deterministic reconciliation reported complete_pair_count = 120; planned_episode_count = 240; completed_episode_count = 240; failed_episode_count = 0; all deterministic consistency checks passed. The run started at 2026-08-29T00:39:14.248966+00:00 and completed at 2026-08-29T00:48:37.619198+00:00 (execution manifest timestamps).

Scoring and analysis. Each paired observation produced a binary semantic outcome (pass = emulator-observed final_state satisfied the task's expected_final_state and required_sequence constraints under the scoring rule full_intent_constraint_validation; fail otherwise). The confirmatory analysis followed the preregistered plan: exact two-sided McNemar test and a paired-bootstrap-percentile 95% CI (10,000 resamples, seed = 1729). The preregistered power calculation assumed baseline failure $\approx$0.40 and targeted an absolute reduction of 0.15; we report this assumption and show how the observed baseline invalidates that power justification.

IV. Results

Execution summary and primary outcome. The run completed without technical exclusions. Condition-level summaries: baseline successes = 120, baseline failures = 0; guarded successes = 120, guarded failures = 0 (success rate = 1.0 for both). The paired contingency table was n_11 = 120, n_10 = 0, n_01 = 0, n_00 = 0. Exact two-sided McNemar p-value = 1.0; paired-difference estimate = 0.00. The preregistered paired-bootstrap-percentile 95% CI (10,000 resamples, seed = 1729) = [0.00, 0.00]. Validator traces record repair_applied = false for every guarded episode; therefore no repair-provider-call artifacts exist.

Per-topology and task-difficulty diagnostics. Successes were uniform across topology classes (Edge: 40/40; Core: 40/40; Leaf–spine: 40/40). Representative difficulty and command-count diagnostics extracted from archived task manifests and validator traces include: required_command_count = 2 for make_before_break uplink switchover tasks, 4 for shutdown→configure→restore patterns, and 6 for paired-link atomic MTU changes. Representative per-episode validator-trace numerics from archived episodes: task-000001 parsed_command_count = 4 and observed_touched_field_count = 3 (required_command_count = 4, level = medium, pattern = shutdown_configure_restore); task-000002 parsed_command_count = 2 and observed_touched_field_count = 2 (required_command_count = 2, level = hard, pattern = make_before_break_uplink_switchover); task-000003 parsed_command_count = 6 and observed_touched_field_count = 4 (required_command_count = 6, level = hard, pattern = paired_link_atomic_mtu_change). In these representative episodes validation_before.valid and validation_after.valid = true and violation_count = 0.

Representative diagnostics and accounting. Archived per-episode scoring artifacts show normalized_response

equaled normalized_reference_answer for sampled episodes, parsed_command_count matched required_command_count, and validator.validation_after.valid = true. Deterministic reconciliation and provider-call audit: planned_episode_count = 240; completed_episode_count = 240; failed_episode_count = 0; complete_pair_count = 120; hosted_model_call_event_count = 120; cache_miss_count = 120; cache_hit_count = 0. These artifacts document that shared generator outputs satisfied the encoded workflow_policies so the guard had no corrective actions to apply.

Statistical interpretation. The confirmatory McNemar test returned p = 1.0 because there were zero discordant pairs (n_10 = n_01 = 0). The preregistered bootstrap CI is degenerate at [0.00, 0.00] under the resampling scheme because every resample reproduces the same paired contingency when no discordant pairs exist. The preregistered power calculation assumed baseline failure $\approx$ 0.40; observed baseline = 0.00 violates that assumption and makes the originally targeted effect-size power calculation inapplicable for this execution.

V. Discussion

Interpreting a ceiling outcome. The paired shared-generation design successfully isolates validator/repair marginal effects: because the generator (gpt-5-mini under the frozen prompt strategy) produced valid candidates across the curated task bank, the validator observed zero violations and therefore did not exercise repairs. This ceiling outcome is a valid experimental observation: it documents that, under the declared model/prompt and synthetic corpus, the guard was idle. It does not invalidate prior demonstrations that validators improve safety when generators err; rather it demonstrates how design choices (shared-generation versus independent-condition generation, task selection, prompt calibration) determine whether repair policies can be empirically exercised.

Mechanistic diagnostics and what the artifacts show. The archived validator_trace objects provide machine-verifiable evidence for the mechanistic explanation of the ceiling result. For every guarded episode validator.violation_count = 0, repair_applied = false, validation_after.valid = true, and the shared_initial_candidate text equals the normalized_configuration used for scoring. The provider-call audit (hosted_model_call_event_count = 120, cache_miss_count = 120) and deterministic_reconciliation outputs confirm that the run followed the preregistered shared-initial_generation semantics and that no cross-condition contamination occurred.

Design trade-offs and prespecified recommendations. The main operational lesson is a concrete trade-off between internal validity and the ability to exercise repair behavior. The preregistration anticipated this and recommended alternatives to empirically exercise repairs while preserving some internal control: (1) allow independent generation per condition so guards can intercept generator sampling errors (accepting generator variance in analysis); (2) prespecified controlled prompt perturbations on a subset of tasks to increase generator-induced violation incidence while preserving paired blocks for internal control; (3) task stratification toward higher-complexity or higher-dependency-rule-count cases to raise the probability of generator mistakes; and (4) exploratory arms with an expanded repair budget (>1 repair iteration) to study repair convergence and provider-call costs when repairs are applied. Each recommendation is implementable using the same harness and artifact schema archived with this run.

Operational and auditability implications. Even when unused, an executable deterministic validator that emits structured traces (checked rules, touched fields, violation_codes, repair_applied flags) materially improves post-hoc auditability and failure diagnosis. For operators and researchers the practical implication is twofold: (a) validators produce compact, machine-readable traces that explain why a candidate was accepted or rejected, and (b) paired-shared generation experiments should be accompanied by planned perturbation arms or higher-difficulty strata if the research goal is to quantify repair utility under realistic generator variance.

Threats to validity and limitations (summary). Internal validity in this execution is high because of deterministic provisioning, exhaustive provider-call audits, no technical exclusions, and reproducible validator traces. Construct validity is limited because workflow_policies and task payloads are synthetic encodings that do not capture full vendor heterogeneity or partial telemetry observability; some real-world failure modes may therefore be absent. External validity is constrained by the synthetic deterministic task corpus, a single hosted model (gpt-5-mini), and a frozen prompting strategy. Conclusion validity is affected by the statistical ceiling: the preregistered power calculation assumed baseline failure $\approx$ 0.40; observed baseline = 0.00 invalidates that assumption and renders the McNemar test uninformative for detecting the originally targeted absolute reduction of 0.15. Each of these limitations is reported in the preregistration and archived artifacts.

VI. Limitations

- Confirmatory scope: results are narrowly scoped to the preregistered synthetic task set, a single hosted model (gpt-5-mini), and a frozen prompting strategy; generalization to production heterogeneity is limited.
- Synthetic task provenance: tasks were synthetic/deterministic and may underrepresent noisy, adversarial, or multi-vendor production incidents; external validity to live networks is constrained.
- Shared-generation design: using a single shared initial generation per paired task isolates validator/repair efficacy but precludes detection of generation-variance effects; this choice contributed to the ceiling outcome and limits inference about repair utility on independently generated invalid outputs.
- Validator fidelity: the deterministic emulator/validator is reproducible but simplifies heterogeneity present in multi-vendor, partially observable production environments; some real-world failure modes may not be exposed.
- Statistical ceiling: baseline success reached 100% on the tested task bank, violating the preregistered power

assumptions (targeted absolute reduction = 0.15) and rendering the McNemar confirmatory test uninformative for demonstrating the preregistered effect size.

VII. Conclusion

In a fully preregistered paired experiment (N=120) comparing an executable generate→validate→repair pipeline against an LLM-only workflow under a deterministic harness with gpt-5-mini, every shared initial candidate satisfied the emulator's required_sequence and workflow_policies. The experiment produced n_11 = 120, a paired-difference estimate of 0.00 (exact McNemar p = 1.0; paired-bootstrap 95% CI = [0.00, 0.00], 10,000 resamples, seed = 1729), and validator.repair_applied = false in all guarded episodes. Scientifically, the result documents a ceiling for this curated synthetic task bank and illustrates how a shared-generation paired design isolates validator/repair effects but can mask repair benefits when the generator is well-calibrated for the chosen corpus. Methodologically, we show that preregistered, artifact-rich paired designs combined with executable validators that emit fine-grained traces permit reproducible diagnosis of degenerate confirmatory outcomes and support principled follow-up designs (independent condition generation, prompt perturbations, harder tasks, and expanded repair budgets) to quantify practical repair utility under realistic generator variance.

Disclosure Statement

Mandatory Disclosure Statement: Human role and autonomy boundary — A human Research Director selected the topical family and approved the pre-lock master prompt. After the autonomy lock the registered autonomous system performed literature discovery, preregistration, experiment design, experiment execution, analysis, manuscript drafting, simulated peer review, and revision without manual human edits to technical content. LLM(s) used: gpt-5-mini (hosted). Adapter/orchestration framework and validator: adapter_family = "hosted_netops_gvr_v1"; deterministic task generator = deterministic_netops_task_generator_v5; deterministic emulator validator = deterministic_netops_validator_v5 (validator_version = 5.0). Initial immutable master-prompt reference archived at provenance/master_prompt.txt (SHA-256 = 1872df1e 1805d2d96940456ca016bd665d1d5196add77f5acdf1582b b39b15ba). Preregistration identifier: CNSM2026-A-prereg-v1. Execution accounting: planned_episode_count = 240; completed_episode_count = 240; complete_pair_count = 120; model_calls_used = 120; maximum_model_calls_planned = 240. Stage accounting: a shared_initial_generation stage produced one shared candidate per pair (120 shared generations). Repair policy: guarded repairs were conditional on validator-reported violations (repair_applied only when validator.violation_count>0); validator traces record repair_applied = false for all guarded episodes. No contamination flags or technical exclusions occurred. Representative archival artifacts (task manifests, responses, validator traces, condition summaries, paired contingency table, and deterministic reconciliation) were archived at the time of execution for independent verification. No manual post-lock edits of technical manuscript content were performed.

References

[1] https://arxiv.org/abs/2604.22513
[2] https://arxiv.org/abs/2608.23179
[3] https://doi.org/10.1145/3771567
[4] https://doi.org/10.1145/3603269.3604842
[5] https://arxiv.org/abs/2607.22947
[6] https://arxiv.org/abs/2604.18233
[7] Ioannis Protogeros, Rufat Asadli, Benjamin Hoffman, Laurent Vanbever, "Benchmarking LLM-Driven Network Configuration Repair", 2026
[8] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026